



Navegación Autónoma en Laberintos

Depth-First Search

Alumno: Enrique Lisandro Flores Brito
Profesor: Jorge Hermosillo Valadez
11 de Junio de 2026

Índice de Contenidos

1. Descripción del proyecto	1
1.1. Contexto	1
1.1.1. Visualización	1
1.2. Problema	1
1.2.1. Datos de entrada y salida	1
2. Planteamiento del problema	3
2.1. Definición de un estado del problema	3
2.2. Representación de un estado para el algoritmo de búsqueda	3
3. Metodología	4
4. Resultados	4
4.1. Comportamiento y análisis visual	4
5. Conclusiones	6
Referencias	7

Índice de Figuras

1. A la izquierda: Agente explorando una rama profunda. A la derecha: Agente tras alcanzar con éxito la meta, mostrando la ruta final en rojo.	5
--	---

1. Descripción del proyecto

El presente trabajo aborda la resolución de un problema de búsqueda de rutas en entornos discretos mediante la implementación de un agente inteligente. Específicamente, se desarrolló un algoritmo de Búsqueda Primero en Profundidad (DFS, por sus siglas en inglés) para encontrar un camino válido desde un punto de inicio hasta una meta dentro de un laberinto generado de forma procedimental¹

1.1. Contexto

El problema se origina en el campo de la Inteligencia Artificial, específicamente en el diseño de agentes racionales y algoritmos de búsqueda no informada. La navegación en laberintos es un problema clásico que sirve como abstracción para situaciones más complejas del mundo real, como la planificación de trayectorias en robótica, el enrutamiento de paquetes en redes de computadoras o la resolución de problemas de logística.

En este contexto, un laberinto representa un entorno estático y determinista donde un agente debe tomar decisiones secuenciales (moverse arriba, abajo, izquierda o derecha) para transitar de un estado inicial a un estado objetivo, evitando obstáculos estructurales (paredes).

1.1.1. Visualización

El entorno se modela como una cuadrícula bidimensional (grid) donde las celdas de espacio libre y las paredes determinan la topología del problema. Gráficamente, el algoritmo de generación crea un patrón de pasillos laberínticos mediante la técnica de "búsqueda en profundidad aleatoria" (Randomized DFS), garantizando que siempre exista al menos un camino posible hacia la meta.

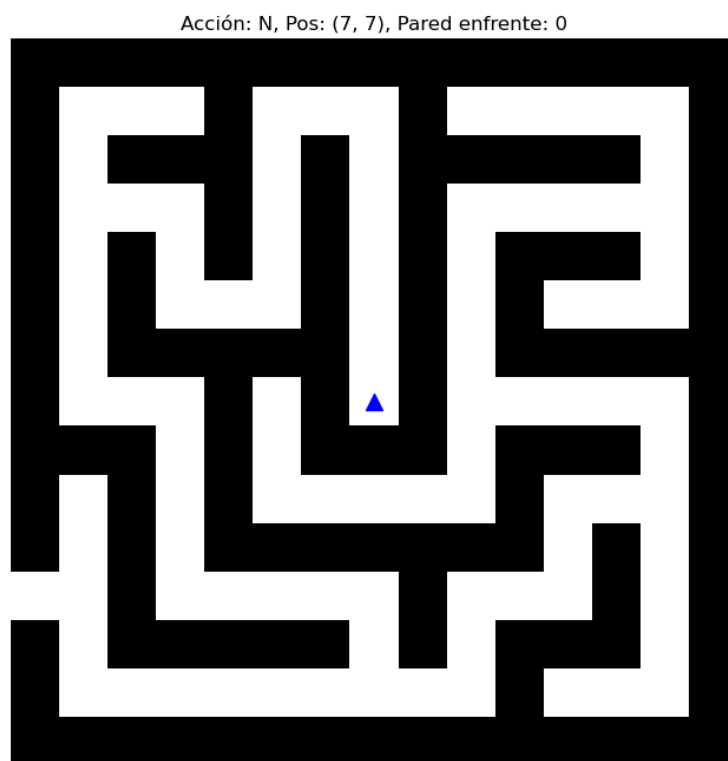
1.2. Problema

Dado un conjunto de datos espaciales representados como una matriz binaria (donde los ceros son caminos libres y los unos son paredes), el problema consiste en determinar una secuencia de acciones direccionales válidas que permitan a un agente transitar desde una coordenada inicial predefinida hasta una coordenada meta.

1.2.1. Datos de entrada y salida

- **Entrada:** Una matriz M de tamaño $(2m + 1) \times (2n + 1)$ representando la topología del laberinto, una tupla de coordenadas de inicio y una tupla de coordenadas meta.
- **Salida:** El algoritmo de búsqueda debe retornar una secuencia (o lista) de acciones discretas (Norte, Sur, Este, Oeste) y los estados (coordenadas) visitados que conforman la ruta solución.

¹ El código fuente completo de la implementación en Python, así como las visualizaciones, se encuentran disponibles en el repositorio: <https://github.com/enriqfl/Navegacion-Autonoma-en-Laberintos>



Representación del laberinto generado. Las áreas oscuras representan paredes y el rastro indica la exploración del agente.

2. Planteamiento del problema

2.1. Definición de un estado del problema

Un estado es una estructura de datos que representa la ubicación del agente dentro del entorno en un momento dado.

Matemáticamente, debido a que el laberinto se mapea sobre una cuadrícula, un estado E se define de manera precisa como una 2-tupla de números enteros que corresponden a los índices de fila y columna de la matriz del entorno:

$$E := (r, c) \text{ con } r, c \in \mathbb{Z}^+$$

El planteamiento parte de un estado inicial $E_0 = (r_{inicio}, c_{inicio})$ situado en el centro geométrico de la matriz, y el objetivo de la búsqueda es alcanzar un estado final $E_f = (r_{meta}, c_{meta})$ ubicado en uno de los bordes externos del laberinto.

2.2. Representación de un estado para el algoritmo de búsqueda

Para que el agente pueda transitar entre estados y el algoritmo explore el árbol de soluciones, se debe formalizar el modelo de transición. A partir de un estado coordinado $E = (r, c)$, el agente cuenta con un espacio de acciones discreto y constante definido por los puntos cardinales:

$$Acciones := \{ 'N', 'S', 'E', 'O' \}$$

La función resultado (*Result*), que genera los estados hijos (sucesores), aplica un operador de desplazamiento sumando vectores direccionales al estado actual, siempre y cuando la celda destino no sea una pared (es decir, que el valor en la matriz sea igual a 0). Matemáticamente, el desplazamiento $\Delta(r, c)$ se rige por:

- $N \rightarrow \Delta = (-1, 0)$
- $S \rightarrow \Delta = (1, 0)$
- $E \rightarrow \Delta = (0, 1)$
- $O \rightarrow \Delta = (0, -1)$

Al evaluar las acciones disponibles en un estado válido, el algoritmo construye iterativamente un árbol de búsqueda (gestionado mediante la clase `Node`), donde cada nodo encapsula el estado actual, una referencia a su nodo padre y la acción que lo originó.

3. Metodología

La estrategia implementada para solucionar el problema es la **Búsqueda Primero en Profundidad** (*Depth-First Search*). Este algoritmo no informado expande siempre el nodo más profundo en la frontera de búsqueda actual. La metodología se desglosa en los siguientes componentes:

- **Estructura LIFO (Pila):** La frontera de exploración se administra utilizando una estructura de datos tipo Pila (*Stack*). Esto asegura que el último nodo descubierto sea el primero en ser expandido, provocando que el agente siga un camino hasta encontrar un callejón sin salida o la meta.
- **Control de estados repetidos (Gráfica):** Para evitar bucles infinitos causados por caminos cíclicos o la reversión inmediata de movimientos (por ejemplo, ir al Norte y luego al Sur repetidamente), se implementó un registro en memoria (conjunto **visited**). Si un estado ya fue explorado, el algoritmo omite su re-expansión.
- **Expansión y Evaluación:** En cada iteración, el agente extrae un nodo de la frontera, lo marca como visitado y verifica si cumple la condición de meta. Si no es la meta, genera todos los estados sucesores válidos (evitando paredes) y los añade a la frontera.
- **Construcción del camino:** Una vez que el estado extraído coincide con el estado meta, el algoritmo recupera la ruta óptima encontrada recorriendo recursivamente los nodos desde el objetivo hasta la raíz (estado inicial) utilizando el atributo **parent** de cada nodo.

4. Resultados

Al ejecutar el modelo utilizando una dimensión paramétrica de $m = 10$ y $n = 10$, el algoritmo de búsqueda DFS logró resolver el laberinto de manera determinista.

4.1. Comportamiento y análisis visual

El rastro de solución fue procesado mediante **matplotlib.animation** para documentar el progreso del agente paso a paso. Se observa cómo el agente toma decisiones basadas en el último nodo disponible, explorando en profundidad cada pasillo antes de hacer un retroceso (*backtracking*) cuando se encuentra de frente con una obstrucción perimetral.

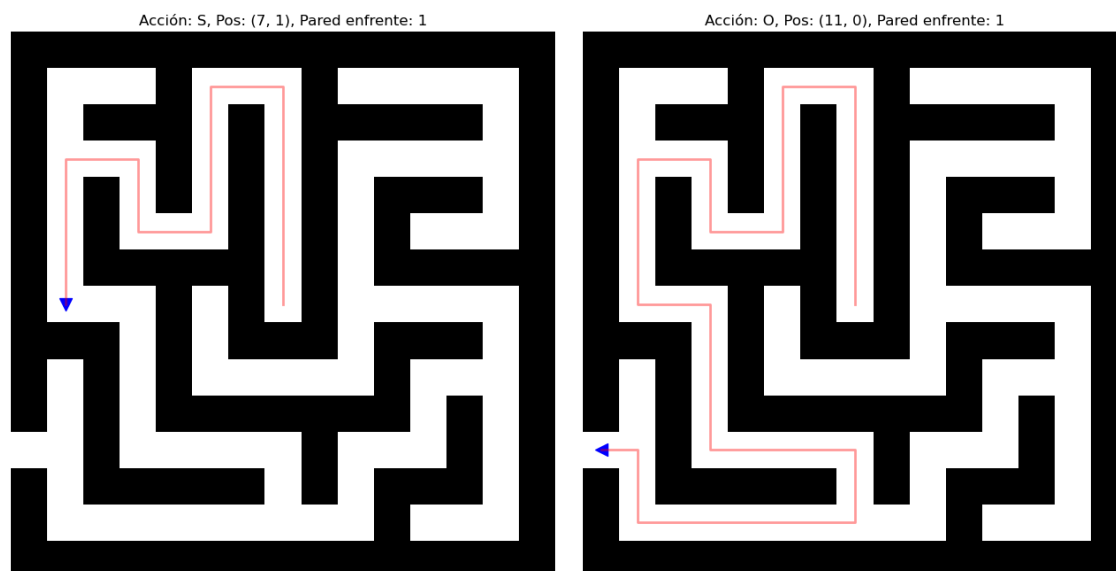


Figura 1: A la izquierda: Agente explorando una rama profunda. A la derecha: Agente tras alcanzar con éxito la meta, mostrando la ruta final en rojo.

5. Conclusiones

El proyecto demuestra empíricamente la efectividad del algoritmo de Búsqueda Primero en Profundidad adaptado a gráficas para la navegación autónoma en entornos discretos. La integración conjunta de un generador de laberintos garantizó escenarios de prueba válidos, mientras que el control algorítmico mediante conjuntos de memoria (**visited**) probó ser estrictamente necesario para dotar al agente de racionalidad básica y prevenir colapsos por ciclos de retroalimentación infinita.

Si bien la estrategia DFS garantiza la completitud en espacios de estados finitos (siempre encontrará la salida si existe), se observó que la ruta generada exhibe el comportamiento característico de no optimidad; es decir, el agente no necesariamente toma el camino más corto, sino el primer camino profundo que lo conecte con la meta. Estos hallazgos sientan las bases para iteraciones futuras donde se podrían implementar heurísticas, como en la búsqueda A*, para optimizar el criterio del costo de la trayectoria.

Referencias

- [1] AIMA Python Contributors, Russell, S., y Norvig, P., “aima-python: Python implementation of algorithms from artificial intelligence: A modern approach”. GitHub repository, 2026, <https://github.com/aimacode/aima-python>.
- [2] Russell, S., Norvig, P., y Intelligence, A., “A modern approach”, Artificial Intelligence. Prentice-Hall, Englewood Cliffs, 1995.